

ClaudeOS — Client Documentation

Version: 17.3

Last Updated: 2026-06-22

Brand: #407E3C Green · White · #5a9e56 Accent

What Is ClaudeOS?

ClaudeOS is an AI Operating System — a secure, multi-user coordination layer that unifies AI agents, memory, workflows, and project management into a single authenticated control center. It runs on your local machine (Windows 10) and exposes a polished dashboard at `http://localhost:8501`.

Getting Access

Admin Creates Your Account

Your admin will create your account and provide: - **Username** and temporary **password** - Your assigned **namespace** (e.g. `reci-transport`, `ivycandy-hair`) - Your **role**: client or viewer

You will be prompted to change your password on first login.

Self-Registration (if enabled)

If self-registration is enabled by the admin: 1. Open the dashboard at `http://localhost:8501` 2. Click the **Register** tab on the login screen 3. Enter username, email, password, and select your namespace 4. Log in separately after registration

Password requirements: minimum 10 characters, at least one uppercase letter, one lowercase letter, and one digit.

Roles & Permissions

Feature	Admin	Operator	Client	Viewer	Staff
All namespaces	■	■	Own only	Own only	—

Memory read	■	■	■	■	—
Memory write	■	■	■	■	—
Run agents	■	■	■	■	—
View outputs	■	■	■	■	—
Manage workflows	■	■	■	■	—
View/work tickets	■	■	■	■	Assigned only
Assign & advance tickets	■	■	■	■	Self-assign ■
Admin Panel	■	■	■	■	■
Observability Dashboard	■	■	■	■	■

Staff is a dedicated support role. Staff users see only tickets assigned to them, can self-assign open tickets, and can advance ticket status. They do not have access to memory, agents, outputs, or admin functions.

Dashboard Pages

Overview

The home page shows: - **System Status** — API and database health at a glance - **KPI Cards** — memory entries, agents, runs, workflows, outputs - **Recent Events** — last 8 agent run results with timestamps and eval score pills - **Quick Dispatch** — run any agent against any namespace with a prompt - **Memory by Namespace** — count of active memory entries per namespace - **Live auto-refresh toggle** — when enabled, the feed refreshes every 8 seconds - **Error alert strip** — red banner displayed when any recent run failed - **Running-now indicator** — yellow banner showing agents currently executing

Eval score pills appear on each run entry, color-coded: - Green — score ≥ 4.0 - Amber — score ≥ 2.5 - Red — score < 2.5

Memory

Browse, search, write, and delete memory entries. - Full-text search across keys and values - Filter by namespace, category, and minimum confidence - Write entries with category (fact, preference, reminder, task), TTL, and confidence score - Reminder entries use a date/time picker for expiry - Search via FTS5 for instant keyword matching - Hybrid BM25+vector search available via the search toggle for higher-recall results - Bulk delete: select entries with checkboxes, then use the bulk toolbar to delete all selected

Agents

The Agents page has three tabs:

Chat tab

Conversational multi-turn chat with any agent. Each session maintains full conversation history per agent. Features: - **Streaming responses** — tokens appear as they arrive, no waiting for the full output. Fully implemented via Server-Sent Events (SSE) with token-by-token rendering. - **Image/screenshot upload** — attach images for visual analysis (e.g. UI screenshots, charts, documents) - **Voice input** — click the microphone button, record audio, and the system auto-transcribes it into the prompt field - **Conversation history** — full back-and-forth exchange shown per agent; **history is restored from the database on every page refresh** — conversations are never lost between sessions - **Agent descriptions in dropdown** — the agent selector shows [CATEGORY] description for each agent so you can choose the right specialist at a glance - **Context degraded warning** — if memory retrieval times out, a subtle caption appears under the affected response noting that business-specific context may be missing - **Clear conversation** — button resets the current conversation and starts a new session - **Eval score badges** — every assistant response shows its quality score after the async eval completes (~10 seconds) - **Token usage** — token count displayed after each response

Every streaming chat run is saved to run history automatically, with output, tokens, and eval score recorded.

Catalog tab

The original agent grid: all 12 registered agents with their descriptions, capabilities, and a quick-run form. Run any agent with a custom prompt and namespace. Live output polling auto-refreshes every 3 seconds until the run completes.

Run History tab

Full run history across all agents. Now includes: - Quality score column with color-coded pill - Eval dimension breakdown (task completion, factual grounding, conciseness, safety) expandable per row -

Per-agent averages in the summary header - Bulk delete run history entries via checkbox selection

Workflows

- View all 7 configured pipelines
- Trigger workflows manually
- See last-run timestamps and schedules
- Monitor pipeline steps and their statuses

Webhooks tab — enable a webhook URL per workflow so external systems can trigger it:

1. Open a workflow and click the **Webhooks** tab
2. Click **Enable Webhook** — a unique URL and secret are generated
3. Copy the webhook URL and secret
4. POST to the URL from any external system with `X-Webhook-Secret: <secret>` to fire the workflow
5. The webhook endpoint requires no JWT — authentication is via the secret header only

Client Vault

- **Namespaces** — view your namespace details, workspace stats (files, KB), and project count
- **Projects** — list projects in your namespace, update status (active/paused/archived)
- **Context Files** — upload per-namespace context files injected into agent system prompts

Outputs

- Browse all AI-generated outputs: reports, summaries, plans, analyses, code
- Full-text search across output content
- Filter by namespace, type, and date
- Export individual outputs as Markdown or PDF
- Bulk delete: select outputs with checkboxes and delete in one action

Tickets

The ticketing system provides a full support and task lifecycle within ClaudeOS.

- **Create tickets** — title, description, priority (P1–P4), and namespace

Priority / SLA tiers:

- P1 — Critical (immediate response required)
- P2 — High
- P3 — Medium
- P4 — Low
- **Status flow** — Open → In Progress → Resolved → Closed
- **Assignment** — admin and operator can assign tickets to staff or other users; staff can self-assign open tickets
- **Comments** — add threaded comments to any ticket; comments are loaded on demand when you open the comment panel
- **Resolution notes** — record resolution details when closing a ticket
- **Bulk delete** — select multiple tickets and delete in one action
- **Stats panel** — ticket counts broken down by status and priority

Observability (admin/operator only)

Four sub-tabs providing full system visibility:

Quality Scores

- Per-agent average eval scores displayed as a ranked table
- Distribution chart showing score spread across all runs
- Dimension breakdown: task completion, factual grounding, conciseness, safety
- Low-quality alert: red warning when any agent's average drops below 2.5

Latency

- p50, p95, and p99 latency across all runs
- Per-agent average latency table sorted by slowest
- Time-series chart showing latency trends over the past 7 days

Token Cost

- Total input and output token counts across all runs
- Estimated USD cost based on current claude-sonnet-4-6 pricing
- Per-agent breakdown: tokens used, cost, run count, avg cost per run

Memory Health

- Entry counts per namespace: total, active, consolidated, expired
- Storage size estimate per namespace
- Manual consolidation trigger button — runs the consolidation job immediately rather than waiting for the scheduled 4-hour window
- Visual indicator when any namespace has not been consolidated in over 24 hours

Settings (Admin/Operator)

- System configuration
- Email notification status and test button
- Supabase sync controls — trigger manual push, view per-table sync state, reset watermarks
- **Sync Log** — view the last 50 sync run records with OK/fail counts and error detail; select entries individually or use **Select All**, then click **Delete Selected (N)** to remove them in one action; a confirmation message persists after delete

Usage (Client/Viewer only)

- **Namespace Pulse Score** — composite 0–100 health gauge: $\text{quality} \times 40\% + \text{ticket resolve} \times 30\% + \text{memory freshness} \times 20\% + \text{workflow health} \times 10\%$
- **How is Pulse Score calculated?** — expandable breakdown with plain-English guidance per dimension
- **KPI grid** — total runs, tokens used, estimated cost, average quality, output count
- **Actionable alerts** — when Pulse Score < 60, targeted alerts appear with a **Go** → button that navigates directly to the relevant page (Agents, Tickets, Memory, or Workflows)
- **Recent activity feed** — last 10 agent runs with status, agent name, and timestamps
- **Memory summary** — total and recently-updated memory entry counts

Admin Panel (Admin only)

- **Users** — create, deactivate, unlock, and reset passwords for all users
 - **API Keys** — create and revoke API keys; raw key shown once on creation
 - **Audit Log** — paginated event log: logins, failures, lockouts, user actions
 - **Sessions** — view active refresh token sessions, revoke any session
 - **Security** — configure lockout threshold, lockout duration, token TTLs, enable/disable self-registration
-

Theme

A dark/light mode toggle sits in the **bottom-left corner** of every page, including the login screen. It is a circular icon button (44px) with no text label: - **Dark mode:** dark navy circle with a white sun icon - **Light mode:** light gray circle with a dark crescent moon icon

Your theme preference persists for your current session.

Security Features

- **JWT-based authentication** — tokens expire after 60 minutes, auto-refreshed silently
 - **Account logout** — 5 failed login attempts triggers a 15-minute logout (admin-configurable)
 - **Case-insensitive login** — usernames are matched case-insensitively (`Romanus`, `ROMANUS`, and `romanus` all work)
 - **Audit log** — every login, logout, failure, user action, and password change is logged with IP and timestamp
 - **Namespace isolation** — clients can only read and write within their assigned namespace
 - **Bcrypt password hashing** — 12 rounds, never stored in plaintext
 - **Forced password change** — admin can flag accounts requiring a new password on next login
-

Quality Scoring

Every agent run is automatically scored by Claude Haiku (LLM-as-Judge) on 4 dimensions. Scoring is fully asynchronous — it runs in the background after the run completes and does not add latency to the agent response.

Dimension	Scale	Weight	What it measures
Task Completion	0–5	40%	Did the output address the prompt fully?
Factual Grounding	0–5	30%	Are claims grounded in injected memory context?

Conciseness	0–5	20%	Appropriate length, no padding or repetition?
Safety	pass/fail	10%	No harmful, biased, or dangerous content?

Overall score = weighted average of the four dimensions. A safety fail caps the overall score at 1.0 regardless of other scores.

Scores appear: - In **Run History** as a colored pill next to each run - In **Overview** as a pill on each recent event entry - In **Chat** as a badge below each assistant response (appears ~10 seconds after the response completes) - In **Observability** → **Quality Scores** as aggregated analytics

Low-quality threshold: runs scoring below 2.5 are flagged. The Observability dashboard alerts when any agent's rolling average drops below this threshold.

Advanced Memory

Hybrid BM25 + Vector Search

Memory search uses two retrieval methods simultaneously: - **BM25** — keyword-based scoring; excellent for exact terms and technical queries - **Vector (ChromaDB)** — semantic similarity; excellent for concept-level retrieval

Results from both are fused using Reciprocal Rank Fusion (RRF) before being returned. Enable hybrid search via the toggle on the Memory page for higher-recall results.

Memory Consolidation

An automated job runs every 4 hours: 1. Groups semantically similar memory entries per namespace 2. Calls Claude to synthesize each cluster into a single, more concise summary entry 3. Marks the originals as consolidated (they are archived, not deleted)

This keeps memory stores lean and coherent over time. You can also trigger consolidation immediately from **Observability** → **Memory Health**.

Tiered Context Injection

When an agent runs, only the most relevant memory entries are injected into its system prompt, ranked by relevance to the current query. This reduces token consumption by ~40% while maintaining context quality.

MCP Integration (Developers)

ClaudeOS exposes all 12 agents as MCP (Model Context Protocol) tools. Any MCP-compatible client — Claude Desktop, Cursor, custom agents — can call your ClaudeOS agents as first-class tools without writing any custom integration code.

Start the MCP server:

```
.\scripts\start_mcp.ps1
```

The server starts on port 5100. Verify it is running:

```
curl http://localhost:5100/mcp
```

Add to Claude Desktop's config (%APPDATA%\Claude\claude_desktop_config.json):

```
{
  "mcpServers": {
    "claudeos": {
      "url": "http://localhost:5100/mcp"
    }
  }
}
```

Restart Claude Desktop. All 12 agents will appear as available tools in the tool picker.

Install optional dependencies first if not already present:

```
pip install mcp uvicorn
```

A2A Agent Cards

Each agent also exposes a machine-readable capability card for agent-to-agent discovery:

```
GET http://localhost:5000/api/v1/agents/<name>/.well-known/agent.json
```

The card describes the agent's name, description, accepted input schema, output schema, and streaming/multi-turn capabilities. External orchestrators can use this endpoint to automatically discover and delegate to ClaudeOS agents.

Webhook API (Developers)

Trigger workflows from external systems without a JWT token.

Step 1 — Enable the webhook

Open the Workflows page, select a workflow, click the **Webhooks** tab, and click **Enable Webhook**. A unique URL and HMAC secret are generated and displayed. Copy both.

Step 2 — Fire the workflow

```
curl -X POST http://localhost:5000/api/v1/workflows/morning-briefing/trigger \
-H "X-Webhook-Secret: your-secret" \
-H "Content-Type: application/json" \
-d '{"context": {"namespace": "global", "topic": "AI news"}}'
```

The `context` object is passed as input to the first pipeline step. The webhook endpoint is public (no JWT required) — authentication is via the `X-Webhook-Secret` header only.

Step 3 — Verify execution

The response returns the run ID. Poll `GET /api/v1/agents/runs/<id>` to check status, or view the run in the **Run History** tab.

SSE Streaming API (Developers)

Stream agent responses token-by-token via Server-Sent Events:

```
curl -N "http://localhost:5000/api/v1/agents/writing-agent/stream?prompt=Hello&namespace=global" \
-H "X-API-Key: your_api_key"
```

Event types:

Event type	Payload	Description
token	<code>{"type": "token", "text": "..."} </code>	One text chunk as it arrives
done	<code>{"type": "done", "run_id": "...", "tokens_in": N, "tokens_out": N} </code>	Stream complete; includes run ID and token counts
error	<code>{"type": "error", "message": "..."} </code>	An error occurred; run marked as failed

context_degraded	<code>{"type": "context_degraded"}</code>	Memory retrieval timed out; agent used fast fallback context — response may lack business-specific detail
------------------	---	--

Every stream run is saved to the database automatically — the run appears in Run History with `status=done`, output, and token counts recorded.

API Access (Developers)

The REST API runs at `http://localhost:5000/api/v1/`.

Authenticate with JWT:

```
# Login
curl -X POST http://localhost:5000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"your_user","password":"your_pass"}'

# Use token
curl http://localhost:5000/api/v1/agents \
-H "Authorization: Bearer <access_token>"
```

Or use an API key (scripts/automation):

```
curl http://localhost:5000/api/v1/memory \
-H "X-API-Key: your_api_key"
```

Key Endpoints

Method	Endpoint	Description
POST	<code>/auth/login</code>	Login, get access + refresh tokens
POST	<code>/auth/refresh</code>	Renew access token using refresh token
POST	<code>/auth/logout</code>	Revoke current session
GET	<code>/auth/me</code>	Current user info
GET	<code>/memory</code>	List memory entries

POST	/memory	Write a memory entry
POST	/memory/consolidate	Trigger memory consolidation immediately
GET	/memory/hybrid-search	Hybrid BM25+vector search with RRF reranking
GET	/agents	List all agents
POST	/agents/{name}/run	Dispatch agent run (async, returns run_id)
GET	/agents/runs/{id}	Poll run status and output
GET	/agents/runs	List all runs (filterable by namespace/status)
GET	/agents/{name}/stream	SSE streaming response (token-by-token)
GET	/agents/{name}/.well-known/agent-2400	Agent Card
GET	/agents/{name}/conversations	List multi-turn conversation sessions
GET	/agents/{name}/conversations/turns	Get most recent turns for latest conversation (used for history restore on page refresh)
POST	/agents/{name}/runs/{id}/cancel	Cancel a pending/running run
GET	/outputs	List outputs
DELETE	/outputs/bulk	Bulk delete outputs
GET	/tickets	List tickets
POST	/tickets	Create a ticket
GET	/tickets/{id}	Get ticket detail
PUT	/tickets/{id}	Update ticket (status, assignee, resolution)
DELETE	/tickets/{id}	Delete a ticket

DELETE	/tickets/bulk	Bulk delete tickets
GET	/tickets/stats	Ticket counts by status and priority
GET	/workflows	List workflows
POST	/workflows/{name}/run	Trigger workflow manually
POST	/workflows/{name}/trigger	Webhook trigger (public, X-Webhook-Secret auth)
POST	/workflows/{name}/webhook/enable	Enable webhook + generate secret
GET	/system/status	System health detail (auth required)
GET	/health	Public health check (no auth)

Starting the System

```
.\scripts\start.ps1
```

This kills any existing processes on :5000 and :8501, starts the Flask API via waitress, verifies /health, then starts the Streamlit dashboard.

Start MCP server (optional):

```
.\scripts\start_mcp.ps1
```

First-time setup:

```
pip install -r requirements.txt
python scripts/migrate.py
python scripts/seed_agents.py
python scripts/seed_workflows.py
python scripts/seed_namespaces.py
python scripts/create_admin.py --username admin --password Admin123!
.\scripts\start.ps1
```

Cloud Sync (Supabase)

Outputs and memory can be synced to Supabase for cloud backup and sharing.

1. Create a [Supabase](#) project
2. Run `sync/supabase_schema.sql` in the Supabase SQL Editor

Add credentials to `.env`:

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_KEY=your_service_role_key
```

4. Restart the server — auto-sync fires every 15 minutes
5. Or trigger manually from the **Settings** tab

Troubleshooting

Problem	Solution
Can't reach dashboard	Run <code>.\scripts\start.ps1</code> — check both ports are alive
Login fails	Check username/password (login is case-insensitive); after 5 failures wait 15 min or ask admin to unlock
"Must change password" shown	Enter current password + new password in the prompt
API returns 401	Token expired — log out and log back in
Output won't export	Try Markdown export; PDF may need wkhtmltopdf installed
Sync fails	Verify <code>SUPABASEURL</code> and <code>SUPABASESERVICE_KEY</code> in <code>.env</code> ; restart after changes
Ticket comments not loading	Click the "■ Comments" toggle — comments load on demand
Eval scores not appearing	Scores are async — wait ~10s after run completes; check API logs for Haiku errors

Streaming returns error	Ensure only one Flask process is on port 5000; restart with <code>.\scripts\start.ps1</code>
Voice input not working	Run <code>pip install openai-whisper</code> ; first use downloads ~140MB model automatically
MCP server not found	Run <code>pip install mcp uvicorn</code> then <code>.\scripts\start_mcp.ps1</code>
Runs stuck as "running"	Restart server; stuck runs from crashed sessions are auto-cleaned on next health check
Agents not responding	Check <code>ANTHROPIC_API_KEY</code> in <code>.env</code> ; verify it is not expired or rate-limited
ChromaDB slow on first start	Normal — sentence-transformers model loads on first request (~30-60s cold start)
Settings shows "rate-limited" toast	Too many rapid page loads hit the sync status endpoint; wait a few seconds and refresh
Settings shows "Could not fetch sync status"	Supabase sync status fetch failed — check <code>.env</code> <code>SUPABASE_URL</code> ; if not configured, that section is expected
Sync log Select All not working	Reload the Settings page and try again; if persists, hard-refresh the browser (Ctrl+Shift+R)
Chat history missing after refresh	Check that Flask API is running; history loads from DB on first render of each agent/namespace pair
"Context unavailable" caption on response	Memory retrieval timed out for that response; subsequent messages will have full context once ChromaDB recovers

Support

Contact your admin (Romanus Igwe) or open an issue in the project repository.

Stack versions: Python 3.11+ · Flask 3.0 · Streamlit 1.38 · SQLite FTS5 · ChromaDB · claude-sonnet-4-6 · Claude Haiku (eval) · rank-bm25 · plotly · waitress (WSGI)